📘 GUÍA DE LABORATORIO PRÁCTICO

Manual de Ejemplos a Realizar en JavaScript

Guía técnica integral con los 12 ejemplos del manual de teoría: enunciados detallados, objetivos didácticos, desarrollo paso a paso y código completo con consola interactiva para el dominio de los objetos en notación literal.

🧱 Bloque 1: Fundamentos y Sintaxis

Ejemplos 01 al 04: Creación, anatomía, tipos de datos, claves dinámicas, mutación en memoria y operador delete.

⚙️ Bloque 2: Métodos y Colecciones

Ejemplos 05 al 08: Contexto this, estructuras anidadas, arrays de objetos homogéneos e introspección de claves.

🧬 Bloque 3: Memoria y ES6+

Ejemplos 09 al 10: Referencias frente a copias (shallow vs deep con structuredClone) y desestructuración moderna con spread.

🛡️ Bloque 4: Control y Persistencia

Ejemplos 11 al 12: Getters y setters con validación, privacidad real con clausuras, serialización JSON y localStorage.

Ramón Abramo

Profesor de Desarrollo Web y Programación Frontend · Santander (Cantabria)

Certificado de Profesionalidad IFCD0110

Edición 2026 · Soluciones Paso a Paso Comentadas

📑 ÍNDICE DE LOS 12 EJEMPLOS A REALIZAR

(Haz clic en cualquier ejemplo para navegar a su desarrollo)

01. Creación y Anatomía de Objetos	Pág. 3	02. Tipos de Valores y Nombres de Claves	Pág. 4
03. Lectura y Acceso a Propiedades	Pág. 5	04. Modificar, Añadir y Borrar (delete)	Pág. 6
05. Métodos y la Palabra Clave 'this'	Pág. 7	06. Objetos Anidados y Encadenamiento ?.	Pág. 8
07. Arrays de Objetos y Métodos Funcionales	Pág. 9	08. Recorrer e Inspeccionar Objetos	Pág. 10
09. Referencias y Copias (structuredClone)	Pág. 11	10. Desestructuración y Operador Spread	Pág. 12
11. Getters, Setters y Privacidad (Closures)	Pág. 13	12. Serialización JSON y LocalStorage	Pág. 14

🎯 Metodología Didáctica del Manual de Ejemplos

Este manual acompaña al alumno en el desarrollo práctico de cada uno de los conceptos explicados en la teoría. Cada página corresponde a un ejemplo autónomo, estructurado en tres bloques clave:

1. Enunciado y Objetivos

Define con claridad qué entidad se va a modelar, qué problema resuelve y qué métodos o sintaxis específica de JavaScript se ejercitan.

2. Desarrollo Paso a Paso

Desglosa en fases lógicas la escritura del código: desde la declaración inicial de datos hasta la interacción con la consola y la interfaz HTML.

3. Código Completo y Salida de Ejecución

Incluye el script `app.js` terminado con comentarios didácticos línea por línea y la salida esperada en la consola del navegador (F12) o consola visual.

> 💡 **Dónde encontrar el código fuente ejecutable:**
> Todos los ejemplos están implementados como mini-aplicaciones listas para abrir en el navegador dentro de la carpeta `ejercicio_18/ejemplos/`. Puedes abrir sus archivos `index.html` directamente o inspeccionar sus scripts `app.js`.

🧱 Ejemplo 01: Creación y Anatomía de Objetos Literales

ENUNCIADO: Crear un objeto literal que modele la entidad `coche` con propiedades de marca, número de puertas y un método de acción `arrancar()`. Posteriormente, demostrar las tres formas habituales de instanciación literal en JavaScript (directa, creación vacía con asignación posterior y sintaxis abreviada ES6).

🧭 Desarrollo Paso a Paso:

1. **Paso 1:** Declarar la constante `coche` utilizando llaves `{}` y pares `clave: valor`. Definir el método abreviado `arrancar() { return "En marcha"; }`.
2. **Paso 2:** Leer los datos por consola con `coche.marca` y ejecutar el método con paréntesis: `coche.arrancar()`.
3. **Paso 3:** Implementar la forma vacía `const usuario = {};` y poblarla asignando `usuario.nombre = "Marcos"`.
4. **Paso 4:** Implementar la sintaxis abreviada ES6: variables `marca` y `modelo` agrupadas directamente como `{ marca, modelo }`.

💻 Código Fuente Completo (app.js):

```
// 1. Anatomía básica de un objeto literal (Pág. 4 del manual)
const coche = {
  marca: "Toyota",
  puertas: 5,
  arrancar() {
    return "En marcha";
  }
};

console.log("Marca:", coche.marca);         // "Toyota"
console.log("Puertas:", coche.puertas);     // 5
console.log("Estado:", coche.arrancar());   // "En marcha"

// 2. Tres formas de crear un objeto literal (Pág. 5 del manual)
// Forma 1: Directa
const usuarioDirecto = { nombre: "Ana", rol: "Alumna" };

// Forma 2: Vacío y asignación posterior
const usuarioVacio = {};
usuarioVacio.nombre = "Marcos";
usuarioVacio.rol = "Profesor";

// Forma 3: Abreviada ES6 (variables con mismo nombre que la clave)
const marca = "Logitech";
const modelo = "MX Master 3";
const raton = { marca, modelo }; // { marca: "Logitech", modelo: "MX Master 3" }
```

💡 Salida esperada en Consola:
Marca: Toyota | Puertas: 5 | Estado: En marcha
Forma 3 (Abreviada ES6): { marca: "Logitech", modelo: "MX Master 3" }

🏷️ Ejemplo 02: Tipos de Valores y Nombres de Claves

ENUNCIADO: Diseñar un objeto polivalente que almacene todos los tipos de datos primitivos y complejos admitidos en JavaScript. Experimentar con claves que contengan espacios y guiones mediante comillas, y crear propiedades computadas dinámicamente mediante la sintaxis `[expresión]`.

🧭 Desarrollo Paso a Paso:

1. **Paso 1:** Modelar el objeto `datosDiversos` incluyendo string, number, boolean, array, sub-objeto y función miembro.
2. **Paso 2:** Declarar claves especiales que no sean identificadores válidos: `"código postal"` y `"fecha-alta"`. Observar la obligatoriedad de entrecomillarlas.
3. **Paso 3:** Declarar una variable `let tipoContacto = "email"` y definir una clave computada entre corchetes dentro del literal: `[tipoContacto]: "contacto@santander.es"`.
4. **Paso 4:** Acceder a las claves especiales obligatoriamente mediante corchetes: `objeto["código postal"]`.

💻 Código Fuente Completo (app.js):

```
const claveDinamica = "idioma_preferido";

const perfilUsuario = {
  // Tipos de datos variados:
  id: 1042,
  nombre: "Beatriz Sáez",
  activo: true,
  habilidades: ["HTML5", "CSS3", "JavaScript"],
  ubicacion: { ciudad: "Santander", provincia: "Cantabria" },

  // Claves especiales con espacios o guiones (requieren comillas obligatorias):
  "codigo postal": "39001",
  "tipo-contrato": "Indefinido",

  // Clave computada a partir de una variable externa:
  [claveDinamica]: "es-ES"
};

// Lectura de claves estándar:
console.log(perfilUsuario.nombre);            // "Beatriz Sáez"

// Lectura de claves con caracteres especiales (¡OBLIGATORIO CORCHETES!):
console.log(perfilUsuario["codigo postal"]);   // "39001"
console.log(perfilUsuario["tipo-contrato"]);   // "Indefinido"
console.log(perfilUsuario[claveDinamica]);     // "es-ES"
```

⚠️ **Regla Sintáctica Fundamental:**
`perfilUsuario.codigo postal` provoca un `SyntaxError` inmediato. Cuando la clave no es un identificador estándar de una sola palabra, la notación de corchetes `objeto["clave con espacio"]` es mandatoria.

🔍 Ejemplo 03: Lectura y Acceso a Propiedades

ENUNCIADO: Comparar la notación de punto frente a la notación de corchetes mediante un selector dinámico de propiedades. Demostrar el resultado al consultar claves inexistentes (`undefined`) y proteger accesos profundos mediante el operador de encadenamiento opcional (`?.`).

🧭 Desarrollo Paso a Paso:

1. **Paso 1:** Crear el objeto `articulo` con propiedades de primer nivel y un objeto anidado `detalles`.
2. **Paso 2:** Probar el acceso por punto: `articulo.precio`.
3. **Paso 3:** Simular una clave seleccionada por el usuario en una variable `let campoBuscado = "stock"` y leerla con `articulo[campoBuscado]`.
4. **Paso 4:** Comprobar qué devuelve leer `articulo.peso` (devuelve `undefined` sin error).
5. **Paso 5:** Intentar leer `articulo.fabricante.telefono` vs `articulo.fabricante?.telefono` para mostrar la protección contra `TypeError`.

💻 Código Fuente Completo (app.js):

```
const articulo = {
  id: 301,
  nombre: "Monitor 27 pulgadas 4K",
  precio: 299.99,
  detalles: {
    panel: "IPS",
    tasaRefresco: "144Hz"
  }
};

// 1. Notación de Punto: directa y limpia
console.log(articulo.nombre);           // "Monitor 27 pulgadas 4K"
console.log(articulo.detalles.panel);   // "IPS"

// 2. Notación de Corchetes: obligatoria si el campo está en una variable
const propiedadConsultada = "precio";
console.log(articulo[propiedadConsultada]); // 299.99

// 3. Propiedades inexistentes: devuelven undefined
console.log(articulo.peso); // undefined (no falla)

// 4. Encadenamiento Opcional (?.) para evitar errores fatales:
// console.log(articulo.fabricante.contacto); // 💥 TypeError: Cannot read properties of undefined
console.log(articulo.fabricante?.contacto);   // undefined (seguro y limpio)
```

💡 Buenas Prácticas Modernas:

Usa siempre el punto `.` por defecto para mantener tu código legible. Reserva los corchetes `[]` para cuando la clave sea dinámica (guardada en una variable) o contenga caracteres reservados.

🖍️ Ejemplo 04: Modificar, Añadir y Borrar Propiedades (CRUD)

ENUNCIADO: Implementar un ciclo completo de mutación sobre un objeto `producto`: actualizar su precio, incorporar una nueva clave de descuento, eliminar de memoria su identificador mediante `delete`, verificar existencia con el operador `in` y blindar el objeto con `Object.freeze()`.

🧭 Desarrollo Paso a Paso:

1. **Paso 1:** Declarar `const producto = { codigo: "P-10", nombre: "Mesa", precio: 120 };`.
2. **Paso 2:** Reasignar el precio asignando un nuevo valor: `producto.precio = 99.95`.
3. **Paso 3:** Añadir la propiedad `stock` que no existía al principio: `producto.stock = 15`.
4. **Paso 4:** Eliminar la clave `codigo` con `delete producto.codigo`.
5. **Paso 5:** Comprobar existencias con `"codigo" in producto` (devolverá `false`).
6. **Paso 6:** Congelar el objeto con `Object.freeze(producto)` y verificar que ya no admite cambios.

💻 Código Fuente Completo (app.js):

```
const producto = {
  codigo: "P-10",
  nombre: "Mesa de Escritorio",
  precio: 120.00
};

// 1. Modificar propiedad existente
producto.precio = 99.95;

// 2. Añadir nueva propiedad en caliente
producto.stock = 15;
producto.color = "Roble";

// 3. Borrar propiedad de memoria con 'delete'
delete producto.codigo;

console.log("Estado tras mutaciones:", producto);
// { nombre: "Mesa de Escritorio", precio: 99.95, stock: 15, color: "Roble" }

// 4. Comprobación con el operador 'in'
console.log("¿Tiene código?:", "codigo" in producto); // false
console.log("¿Tiene stock?:", "stock" in producto);   // true

// 5. Congelar el objeto para hacerlo 100% inmutable
Object.freeze(producto);
producto.precio = 50.00; // Ignorado silenciosamente (o error en modo estricto)
console.log("Precio tras freeze:", producto.precio); // Sigue siendo 99.95
```

⚠️ **Diferencia clave entre const y freeze:**
`const` impide reasignar la variable a otro valor (`producto = {}` falla), pero sus propiedades internas son mutables. Para congelar el contenido interno se utiliza `Object.freeze()`.

⚙️ Ejemplo 05: Métodos y la Palabra Clave 'this'

ENUNCIADO: Crear un objeto `cuenta` con saldo y métodos para `ingresar()` y `retirar()` dinero interactuando con su propio estado mediante `this`. Demostrar empíricamente los dos errores más comunes: el uso de funciones flecha en métodos y la extracción del método a una variable suelta.

🧭 Desarrollo Paso a Paso:

1. **Paso 1:** Definir `cuentaBancaria` con `titular` y `saldo` inicial.
2. **Paso 2:** Implementar los métodos abreviados `ingresar(cantidad)` y `retirar(cantidad)` modificando `this.saldo`.
3. **Paso 3:** Demostrar el fallo de la función flecha: `saludoArrow: () ⇒ this.titular` devolverá `undefined`.
4. **Paso 4:** Demostrar la extracción del método: `const fn = cuenta.ingresar; fn(50)` perderá el contexto de `this`.

💻 Código Fuente Completo (app.js):

```
const cuenta = {
  titular: "Lucía Navarro",
  saldo: 200,

  // Método con sintaxis estándar (enlace correcto de 'this')
  ingresar(cantidad) {
    this.saldo += cantidad;
    return `Ingreso de \${cantidad}€. Saldo actual: \${this.saldo}€`;
  },

  retirar(cantidad) {
    if (cantidad > this.saldo) return "Fondos insuficientes";
    this.saldo -= cantidad;
    return `Retirada de \${cantidad}€. Saldo restante: \${this.saldo}€`;
  },

  // ❌ ERROR CLÁSICO 1: Función flecha (NO tiene su propio this)
  titularArrow: () ⇒ {
    return this.titular; // 'this' apunta a window o global, NO a cuenta
  }
};

console.log(cuenta.ingresar(50));      // Saldo actual: 250€
console.log(cuenta.retirar(100));     // Saldo restante: 150€
console.log(cuenta.titularArrow());   // undefined (¡Fallo de arrow function!)

// ❌ ERROR CLÁSICO 2: Extraer el método fuera del objeto
const funcionSuelta = cuenta.ingresar;
// funcionSuelta(20); // 💥 Error: Cannot read properties of undefined (reading 'saldo')
```

🎯 Regla de Oro del Profesor Ramón:

En métodos de objetos literales, NUNCA uses funciones flecha si necesitas acceder a las propiedades mediante `this`. Usa siempre la sintaxis estándar abreviada `nombreMetodo() { ... }`.

🌳 Ejemplo 06: Objetos Anidados y Encadenamiento Seguro

ENUNCIADO: Modelar la entidad `estudiante` que contenga un objeto anidado para la `direccion` y una colección en array para las `calificaciones`. Implementar un método que calcule la media ponderada e interactuar con subniveles usando encadenamiento opcional (`?.`).

🧭 Desarrollo Paso a Paso:

1. **Paso 1:** Declarar `estudiante` con propiedades primitivas, el sub-objeto `direccion: { ciudad, cp }` y el array `notas: [7, 8.5, 9]`.
2. **Paso 2:** Implementar el método `calcularMedia()` que sume las notas de su propio array con `this.notas.reduce( ... )`.
3. **Paso 3:** Acceder a niveles profundos combinando puntos y corchetes: `estudiante.notas[0]` y `estudiante.direccion.ciudad`.
4. **Paso 4:** Demostrar la consulta de propiedades profundas en objetos que pueden ser nulos mediante `estudiante.tutor?.telefono`.

💻 Código Fuente Completo (app.js):

```
const estudiante = {
  nombre: "David Ruiz",
  modulo: "Desarrollo Frontend",
  direccion: {
    calle: "Paseo Pereda 12",
    ciudad: "Santander",
    cp: "39004"
  },
  notas: [7.5, 8.0, 9.5],

  calcularMedia() {
    const suma = this.notas.reduce((acc, n) => acc + n, 0);
    return (suma / this.notas.length).toFixed(2);
  }
};

// Acceso encadenado por niveles:
console.log("Ciudad:", estudiante.direccion.ciudad); // "Santander"
console.log("Primera nota:", estudiante.notas[0]);   // 7.5
console.log("Nota media:", estudiante.calcularMedia()); // "8.33"

// Acceso seguro con Encadenamiento Opcional (?.):
// Si 'tutor' no existe en el objeto, no lanza error:
const telTutor = estudiante.tutor?.contacto?.telefono;
console.log("Teléfono tutor:", telTutor); // undefined
```

💡 **Protección de Datos Anidados:**
El operador `?.` detiene la evaluación y devuelve `undefined` si la referencia anterior es `null` o `undefined`, evitando la caída catastrófica de tu script.

📋 Ejemplo 07: Arrays de Objetos y Métodos Funcionales

ENUNCIADO: Modelar un catálogo comercial como un array de objetos literales homogéneos. Implementar recorridos con el bucle `for ... of` y aplicar los métodos funcionales más importantes de ES6: `.map()` (transformar), `.filter()` (filtrar), `.find()` (buscar por id) y `.reduce()` (acumular totales).

🧭 Desarrollo Paso a Paso:

1. **Paso 1:** Declarar el array `catalogo` con 4 productos (id, nombre, precio, disponible).
2. **Paso 2:** Recorrer la colección con `for (const prod of catalogo)` e imprimir la ficha de cada artículo.
3. **Paso 3:** Obtener la lista de solo los nombres con `catalogo.map(p ⇒ p.nombre)`.
4. **Paso 4:** Filtrar únicamente los productos disponibles con `catalogo.filter(p ⇒ p.disponible)`.
5. **Paso 5:** Localizar un producto por su clave primaria con `catalogo.find(p ⇒ p.id === 3)`.
6. **Paso 6:** Calcular el valor total del inventario acumulando los precios con `catalogo.reduce()`.

💻 Código Fuente Completo (app.js):

```
const catalogo = [
  { id: 1, nombre: "Teclado Mecánico", precio: 60.00, disponible: true },
  { id: 2, nombre: "Ratón Óptico",     precio: 25.00, disponible: false },
  { id: 3, nombre: "Auriculares USB",  precio: 45.00, disponible: true },
  { id: 4, nombre: "Alfombrilla XXL",  precio: 15.00, disponible: true }
];

// 1. Recorrido con for ... of
for (const prod of catalogo) {
  console.log(`[\${prod.id}] \${prod.nombre} - \${prod.precio}€`);
}

// 2. Transformar con .map(): array de nombres
const nombres = catalogo.map(p ⇒ p.nombre);

// 3. Filtrar con .filter(): solo disponibles
const enStock = catalogo.filter(p ⇒ p.disponible);

// 4. Buscar con .find(): elemento exacto por ID
const articulo3 = catalogo.find(p ⇒ p.id === 3);

// 5. Acumular con .reduce(): suma total de precios
const totalValor = catalogo.reduce((acum, p) ⇒ acum + p.precio, 0);
console.log("Valor total almacén:", totalValor + "€"); // 145€
```

💡 **La Estructura Reina de la Web:**
El 95% de los datos recibidos desde APIs REST o bases de datos (fetch) se estructuran exactamente como un array de objetos literales.

🔄 Ejemplo 08: Recorrer e Inspeccionar Objetos

ENUNCIADO: Demostrar que los objetos literales no son iterables directamente con `for ... of`. Aplicar las herramientas de introspección estándar: el bucle `for ... in` y los métodos estáticos modernos `Object.keys()`, `Object.values()` y `Object.entries()`.

🧭 Desarrollo Paso a Paso:

1. **Paso 1:** Definir el objeto de configuración: `const config = { tema: "oscuro", idioma: "es", alertas: true, zoom: 100 };`.
2. **Paso 2:** Recorrer con el bucle clásico `for (const clave in config)` accediendo al valor mediante corchetes `config[clave]`.
3. **Paso 3:** Obtener la lista de nombres de claves con `Object.keys(config)` y comprobar cuántas propiedades tiene con `.length`.
4. **Paso 4:** Obtener directamente los valores con `Object.values(config)`.
5. **Paso 5:** Recorrer parejas clave-valor con `Object.entries(config)` desestructurando `[clave, valor]`.

💻 Código Fuente Completo (app.js):

```
const configuracion = {
  tema: "oscuro",
  idioma: "es-ES",
  notificaciones: true,
  fuenteSize: 16
};

// 1. Bucle clásico for ... in
for (const clave in configuracion) {
  console.log(`Clave: \${clave} | Valor: \${configuracion[clave]}`);
}

// 2. Object.keys(): array con las claves (permite saber cuántas hay con .length)
const claves = Object.keys(configuracion);
console.log("Total de propiedades:", claves.length); // 4

// 3. Object.values(): array directo con los valores
const valores = Object.values(configuracion);
console.log("Valores guardados:", valores);

// 4. Object.entries(): array de parejas [clave, valor]
for (const [k, v] of Object.entries(configuracion)) {
  console.log(`Config → \${k}: \${v}`);
}
```

💡 ¿Cuándo usar cada herramienta?

Usa `Object.keys()` para comprobar si un objeto está vacío (`Object.keys(obj).length === 0`) y `Object.entries()` para transformar objetos en mapas o listas clave-valor.

🧬 Ejemplo 09: Referencias frente a Copias de Objetos

ENUNCIADO: Demostrar que los objetos se asignan por referencia en memoria RAM y no por valor. Comparar el comportamiento de la asignación directa (`b = a`), la copia superficial con Spread (`{ ...a }`) y la copia profunda integral con `structuredClone(a)` .

🧭 Desarrollo Paso a Paso:

1. **Paso 1:** Crear el objeto base `original = { nombre: "Ana", direccion: { ciudad: "Santander" } }` .
2. **Paso 2:** Asignar `copiaFalsa = original` . Modificar `copiaFalsa.nombre` y comprobar que `original.nombre` muta.
3. **Paso 3:** Realizar copia superficial `copiaSpread = { ...original }` . Modificar el nombre (es independiente), pero al modificar `copiaSpread.direccion.ciudad` muta el original.
4. **Paso 4:** Aplicar `copiaProfunda = structuredClone(original)` y demostrar que ambos objetos son 100% independientes en todos los niveles.

💻 Código Fuente Completo (app.js):

```
const crearBase = () => ({
  nombre: "Ana López",
  direccion: { ciudad: "Santander" }
});

// 1. ASIGNACIÓN POR REFERENCIA (b = a): NO copia nada
const orig1 = crearBase();
const ref = orig1;
ref.nombre = "Carlos";
console.log(orig1.nombre); // "Carlos" 💥 ¡HA MUTADO! (Mismo espacio en memoria)

// 2. COPIA SUPERFICIAL ({ ...a }): Duplica solo el 1er nivel
const orig2 = crearBase();
const spread = { ...orig2 };
spread.nombre = "Beatriz";           // ✅ orig2.nombre sigue siendo "Ana López"
spread.direccion.ciudad = "Laredo";  // 💥 orig2.direccion.ciudad cambia a "Laredo"

// 3. COPIA PROFUNDA REAL CON structuredClone()
const orig3 = crearBase();
const clon = structuredClone(orig3);
clon.direccion.ciudad = "Torrelavega";
console.log(orig3.direccion.ciudad); // "Santander" (✅ 100% Independiente)
```

⚠️ El Mayor Generador de Bugs en JavaScript:
Creer que `const copia = original` duplica un objeto causa fallos silenciosos graves. Si tu objeto contiene otros objetos o arrays internos, usa siempre `structuredClone()` .

✨ Ejemplo 10: Desestructuración y Operador Spread en Objetos

ENUNCIADO: Aplicar las ventajas de la sintaxis ES6+: extraer variables sueltas desde un objeto de usuario mediante desestructuración, asignar nombres alternativos (alias), proporcionar valores por defecto, separar credenciales mediante el operador Rest (`...resto`) y combinar configuraciones con Spread.

🧭 Desarrollo Paso a Paso:

1. **Paso 1:** Definir el objeto `usuario` con id, nombre, rol, email y token privado.
2. **Paso 2:** Extraer `const { nombre, rol } = usuario;` .
3. **Paso 3:** Extraer con alias y valor por defecto: `const { nombre: nombreCompleto, pais = "España" } = usuario;` .
4. **Paso 4:** Separar el token secreto usando Rest: `const { token, ...perfilPublico } = usuario;` .
5. **Paso 5:** Fusionar dos objetos de configuración con Spread: `const final = { ...defecto, ...personalizado };` .

💻 Código Fuente Completo (app.js):

```
const usuario = {
  id: 101,
  nombre: "Lucía Fernández",
  rol: "Editora",
  tokenSecreto: "xyz987tokenPrivado"
};

// 1. Extracción básica de propiedades
const { nombre, rol } = usuario;
console.log(nombre, rol); // "Lucía Fernández" "Editora"

// 2. Alias (renombrar variable) y Valores por Defecto
const { nombre: nombreCompleto, pais = "España" } = usuario;
console.log(nombreCompleto, pais); // "Lucía Fernández" "España"

// 3. Operador Rest ( ... ): Aislar datos sensibles
const { tokenSecreto, ...perfilPublico } = usuario;
console.log(perfilPublico); // { id: 101, nombre: "Lucía Fernández", rol: "Editora" }

// 4. Fusión de objetos con Spread ( ... )
const configDefecto = { tema: "claro", idioma: "es" };
const usuarioConfig = { tema: "oscuro" };
const configFinal = { ...configDefecto, ...usuarioConfig };
console.log(configFinal); // { tema: "oscuro", idioma: "es" }
```

💡 **Patrón en Funciones:**
En JavaScript moderno, es común desestructurar en los propios parámetros: `function render({ nombre, rol }) { ... }` , pasando el objeto completo directamente.

🛡️ Ejemplo 11: Getters, Setters y Privacidad con Clausuras

ENUNCIADO: Diseñar un objeto con propiedades computadas (`get`) y asignación controlada mediante validaciones estrictas (`set`). Implementar una función factoría con clausura (Closure) para blindar datos privados e impedir el acceso exterior a propiedades críticas.

🧭 Desarrollo Paso a Paso:

1. **Paso 1:** Crear el objeto `producto` con `_precioBase` y `_iva`.
2. **Paso 2:** Definir el getter `get precioFinal()` que calcule el PVP dinámicamente sin paréntesis.
3. **Paso 3:** Definir el setter `set precioBase(valor)` que valide que el valor sea numérico y positivo, lanzando un `Error` si no cumple.
4. **Paso 4:** Crear la factoría `crearCuenta(titular, saldoInicial)` con una variable `let saldoPrivado` que solo sea accesible mediante métodos públicos autorizados.

💻 Código Fuente Completo (app.js):

```
const producto = {
  _precioBase: 80,
  _iva: 0.21,

  // Getter: se accede como propiedad normal: producto.precioFinal
  get precioFinal() {
    return Number((this._precioBase * (1 + this._iva)).toFixed(2));
  },

  // Setter: valida antes de permitir la asignación
  set precioBase(nuevo) {
    if (typeof nuevo !== "number" || nuevo <= 0) {
      throw new Error("El precio debe ser un número positivo mayor que 0");
    }
    this._precioBase = nuevo;
  }
};

console.log("PVP:", producto.precioFinal); // 96.80
producto.precioBase = 100;
console.log("Nuevo PVP:", producto.precioFinal); // 121.00

// 2. PRIVACIDAD REAL MEDIANTE FACTORÍAS Y CLAUSURAS (CLOSURES)
function crearCuentaBlindada(titular, saldoInicial) {
  let saldoPrivado = saldoInicial; // Variable inaccesible desde el exterior

  return {
    titular,
    consultarSaldo() { return saldoPrivado; },
    ingresar(cant)   { if (cant > 0) saldoPrivado += cant; }
  };
}

const miCuenta = crearCuentaBlindada("Ramón", 1000);
console.log(miCuenta.saldoPrivado); // undefined (¡Protegido!)
console.log(miCuenta.consultarSaldo()); // 1000
```

🔒 Verdadera Privacidad en JS:
En objetos literales todo es público. La convención con guion bajo (`_saldo`) es solo un aviso visual. Para blindar un dato de accesos indebidos se debe encapsular dentro de una clausura léxica.

💾 Ejemplo 12: Serialización JSON y Persistencia en LocalStorage

ENUNCIADO: Demostrar la serialización bidireccional entre objetos JavaScript en memoria RAM y cadenas de texto plano JSON mediante `JSON.stringify()` y `JSON.parse()`. Analizar los datos no serializables y persistir un estado completo de usuario en la API `localStorage` del navegador.

🧭 Desarrollo Paso a Paso:

1. **Paso 1:** Crear un objeto `perfil` con propiedades estructuradas y métodos.
2. **Paso 2:** Serializar a cadena de texto con `JSON.stringify(perfil, null, 2)` y observar la omisión de funciones.
3. **Paso 3:** Reconstruir el objeto desde el texto JSON con `JSON.parse()`.
4. **Paso 4:** Almacenar el string resultante en el navegador: `localStorage.setItem("usuario_curso", json)`.
5. **Paso 5:** Recuperar el texto de `localStorage.getItem( ... )` y parsearlo de vuelta a objeto funcional.

💻 Código Fuente Completo (app.js):

```
const perfilUsuario = {
  id: 42,
  nombre: "Elena Castro",
  preferencias: { tema: "oscuro", notificaciones: true },
  saludar() { return "Hola"; } // ❌ Se descarta al pasar a JSON
};

// 1. De Objeto en RAM a Texto Plano (Serialización)
const textoJSON = JSON.stringify(perfilUsuario, null, 2);
console.log("Cadena JSON:", textoJSON);

// 2. De Texto Plano a Objeto en RAM (Deserialización)
const objetoReconstruido = JSON.parse(textoJSON);
console.log(objetoReconstruido.nombre); // "Elena Castro"
console.log(objetoReconstruido.saludar); // undefined (las funciones no viajan en JSON)

// 3. Persistencia Real en LocalStorage del Navegador
const CLAVE = "curso_ramon_perfil";

// Guardar (solo acepta cadenas de texto):
localStorage.setItem(CLAVE, JSON.stringify(perfilUsuario));

// Leer al volver a abrir la página:
const datosGuardados = localStorage.getItem(CLAVE);
if (datosGuardados) {
  const perfilCargado = JSON.parse(datosGuardados);
  console.log("Perfil recuperado con éxito:", perfilCargado.preferencias.tema);
}
```

🏆 Resumen Final del Módulo de Objetos:
Dominar la notación literal, el enlace con `this`, las colecciones de objetos, las referencias en memoria y la persistencia con JSON conforma los pilares esenciales de cualquier programador web profesional con JavaScript.